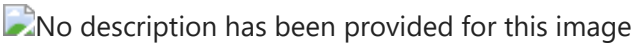


Insurance companies invest a lot of time and money into optimizing their pricing and accurately estimating the likelihood that customers will make a claim. In many countries insurance it is a legal requirement to have car insurance in order to drive a vehicle on public roads, so the market is very large!

(`Source: https://www.accenture.com/_acnmedia/pdf-84/accenture-machine-leaning-insurance.pdf`)

Knowing all of this, On the Road car insurance have requested your services in building a model to predict whether a customer will make a claim on their insurance during the policy period. As they have very little expertise and infrastructure for deploying and monitoring machine learning models, they've asked you to identify the single feature that results in the best performing model, as measured by accuracy, so they can start with a simple model in production.

They have supplied you with their customer data as a csv file called `car_insurance.csv`, along with a table detailing the column names and descriptions below.

The dataset

Column	Description
`id`	Unique client identifier
`age`	Client's age: • `0` : 16-25 • `1` : 26-39 • `2` : 40-64 • `3` : 65+
`gender`	Client's gender: • `0` : Female • `1` : Male
`driving_experience`	Years the client has been driving: • `0` : 0-9 • `1` : 10-19 • `2` : 20-29 • `3` : 30+
`education`	Client's level of education: • `0` : No education • `1` : High school

Column	Description
	• `2` : University
`income`	Client's income level: • `0` : Poverty • `1` : Working class • `2` : Middle class • `3` : Upper class
`credit_score`	Client's credit score (between zero and one)
`vehicle_ownership`	Client's vehicle ownership status: • `0` : Does not own their vehilce (paying off finance) • `1` : Owns their vehicle
`vehcile_year`	Year of vehicle registration: • `0` : Before 2015 • `1` : 2015 or later
`married`	Client's marital status: • `0` : Not married • `1` : Married
`children`	Client's number of children
`postal_code`	Client's postal code
`annual_mileage`	Number of miles driven by the client each year
`vehicle_type`	Type of car: • `0` : Sedan • `1` : Sports car
`speeding_violations`	Total number of speeding violations received by the client
`duis`	Number of times the client has been caught driving under the influence of alcohol
`past_accidents`	Total number of previous accidents the client has been involved in
`outcome`	Whether the client made a claim on their car insurance (response variable): • `0` : No claim • `1` : Made a claim

Random Forest Classification — Required Steps

Overall workflow:

`Explore → Prepare X/y → Preprocess → Split → Pipeline → Random Forest → Train → Predict → Evaluate → Conclusion`

1. **Import the required libraries**

   - Pandas
   - Visualization libraries
   - Scikit-learn preprocessing, pipeline, Random Forest, and evaluation tools

In [1]:
```
# Import required modules
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import randint

# ML libraries
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor, plot_tree
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.metrics import accuracy_score,ConfusionMatrixDisplay,classification_re
from sklearn.ensemble import RandomForestClassifier,RandomForestRegressor

# Start coding!
```

2. **Load the `car_insurance.csv` dataset**

   - Display the first few rows.

In [2]:
```
df = pd.read_csv('car_insurance.csv')
```

In [3]:
```
df.head()
```

Out[3]:

	id	age	gender	driving_experience	education	income	credit_score	vehicle_owne
0	569520	3	0	0-9y	high school	upper class	0.629027	
1	750365	0	1	0-9y	none	poverty	0.357757	
2	199901	0	0	0-9y	high school	working class	0.493146	
3	478866	0	1	0-9y	university	working class	0.206013	
4	731664	1	1	10-19y	none	working class	0.388366	

3. **Explore the dataset**

- Check `shape`, `info()`, and `describe()`.
- Check for missing values.
- Identify numerical and categorical columns.
- Visualize the `outcome` target distribution.

In [4]:
```
print(df.shape)
print(df.info())
```

```
(10000, 18)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 18 columns):
 #   Column               Non-Null Count  Dtype  
---  ------               --------------  -----  
 0   id                   10000 non-null  int64  
 1   age                  10000 non-null  int64  
 2   gender               10000 non-null  int64  
 3   driving_experience   10000 non-null  object 
 4   education            10000 non-null  object 
 5   income               10000 non-null  object 
 6   credit_score         9018 non-null   float64
 7   vehicle_ownership    10000 non-null  float64
 8   vehicle_year         10000 non-null  object 
 9   married              10000 non-null  float64
 10  children             10000 non-null  float64
 11  postal_code          10000 non-null  int64  
 12  annual_mileage       9043 non-null   float64
 13  vehicle_type         10000 non-null  object 
 14  speeding_violations  10000 non-null  int64  
 15  duis                 10000 non-null  int64  
 16  past_accidents       10000 non-null  int64  
 17  outcome              10000 non-null  float64
dtypes: float64(6), int64(7), object(5)
memory usage: 1.4+ MB
None
```

In [5]:
```
df.describe()
```

Out[5]:

	id	age	gender	credit_score	vehicle_ownership	ma
count	10000.000000	10000.000000	10000.000000	9018.000000	10000.000000	10000.0(
mean	500521.906800	1.489500	0.499000	0.515813	0.697000	0.4(
std	290030.768758	1.025278	0.500024	0.137688	0.459578	0.5(
min	101.000000	0.000000	0.000000	0.053358	0.000000	0.0(
25%	249638.500000	1.000000	0.000000	0.417191	0.000000	0.0(
50%	501777.000000	1.000000	0.000000	0.525033	1.000000	0.0(
75%	753974.500000	2.000000	1.000000	0.618312	1.000000	1.0(
max	999976.000000	3.000000	1.000000	0.960819	1.000000	1.0(

In [6]:
```
df.isnull().sum()
```

Out[6]:
```
id                      0
age                     0
gender                  0
driving_experience      0
education               0
income                  0
credit_score          982
vehicle_ownership       0
vehicle_year            0
married                 0
children                0
postal_code             0
annual_mileage        957
vehicle_type            0
speeding_violations      0
duis                    0
past_accidents          0
outcome                 0
dtype: int64
```

In [7]:
```
df['credit_score'].isnull().mean() * 100
```

Out[7]:
```
np.float64(9.82)
```

In [8]:
```
df['annual_mileage'].isnull().mean() * 100 # Will keep both rows because missing va
```

Out[8]:
```
np.float64(9.569999999999999)
```

In [9]:
```
print('Credit Score')
print("Mean:  ", df['credit_score'].mean())
print("Median:", df['credit_score'].median())
print('Annual Mileage')
print("Mean:  ", df['annual_mileage'].mean())
print("Median:", df['annual_mileage'].median())
```

```
Credit Score
Mean:   0.515812809602791
Median: 0.5250327586154788
Annual Mileage
Mean:   11697.003206900365
Median: 12000.0
```

In [10]:
```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy='mean')
df[['credit_score', 'annual_mileage']] = imputer.fit_transform(df[['credit_score',
```

In [11]:
```
df['credit_score'].isnull().sum()
```

Out[11]:
```
np.int64(0)
```

In [12]:
```
df['annual_mileage'].isnull().sum()
```

Out[12]:
```
np.int64(0)
```

In [13]:
```
df.dtypes
```

Out[13]:
```
id                     int64
age                    int64
gender                 int64
driving_experience    object
education             object
income                object
credit_score         float64
vehicle_ownership    float64
vehicle_year          object
married              float64
children             float64
postal_code            int64
annual_mileage       float64
vehicle_type          object
speeding_violations    int64
duis                   int64
past_accidents         int64
outcome              float64
dtype: object
```

In [14]:
```
sns.countplot(df, x='outcome')
plt.title('Distribution of Outcome Target')
plt.xlabel('Outcome (0 = No claim, 1 = Made a claim)')
plt.ylabel('Count')
plt.show()
```

4. **Prepare features and target**

- Create `X` containing the input features.
- Create `y` containing `outcome`.
- Decide whether the `id` column should be removed.

In [15]:
```
X = df.drop(columns=['id', 'outcome', 'postal_code'])
y = df['outcome']
```

5. **Identify feature types**

- Numerical
- Categorical
- Ordinal, where appropriate

In [16]:
```
num_cols = X.select_dtypes(include='number').columns.tolist()
cat_cols = X.select_dtypes(exclude='number').columns.tolist()
```

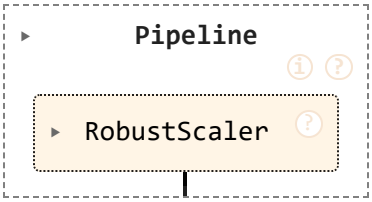
6. **Preprocess the features**

- Apply suitable encoding/transformation.
- Organize preprocessing using `ColumnTransformer`.

In [17]:
```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OrdinalEncoder, RobustScaler
```

In [18]:
```
num_trans = Pipeline(steps=[
    ('scaler', RobustScaler())
])
num_trans
```

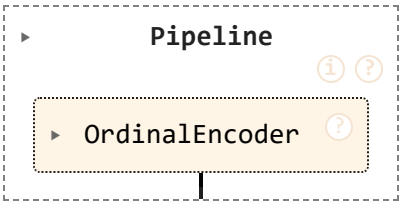
Out[18]:



In [19]:
```
cat_trans = Pipeline(steps=[
    ('ode', OrdinalEncoder())
])

cat_trans
```

Out[19]:

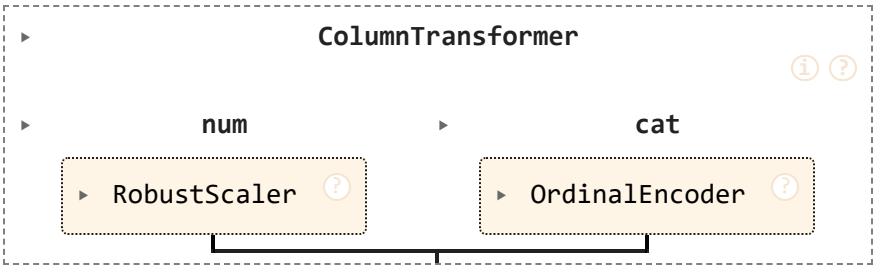


In [20]:
```
from sklearn.compose import ColumnTransformer

preprocessor = ColumnTransformer(
    transformers=[
        ('num', num_trans, num_cols),
        ('cat', cat_trans, cat_cols)
    ]
)

preprocessor
```

Out[20]:



7. **Split the data**

- 80% training data.
- 20% testing data.
- Use `random_state`.

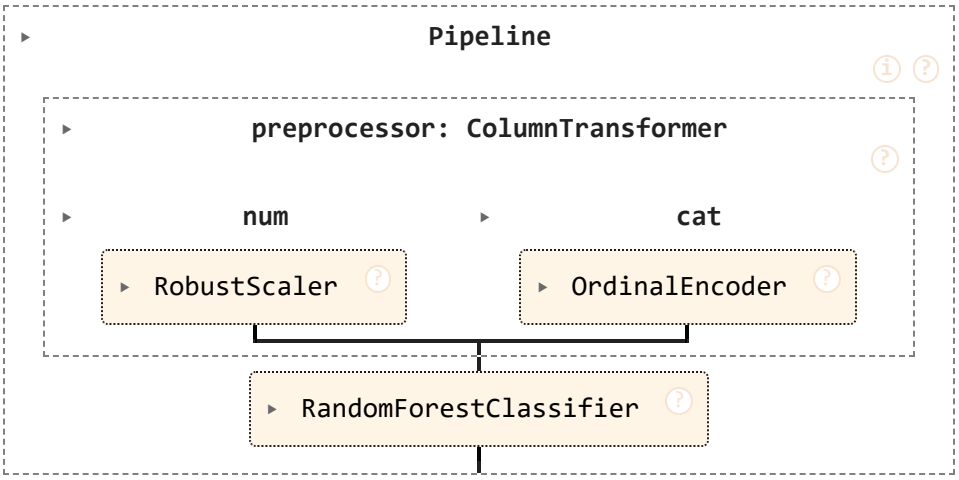
In [21]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=12
)
```

8. **Create a machine learning `Pipeline`**

- Combine the `ColumnTransformer` preprocessing with `RandomForestClassifier`.

In [22]:
```
full_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', RandomForestClassifier())
])
full_pipeline
```

Out[22]:

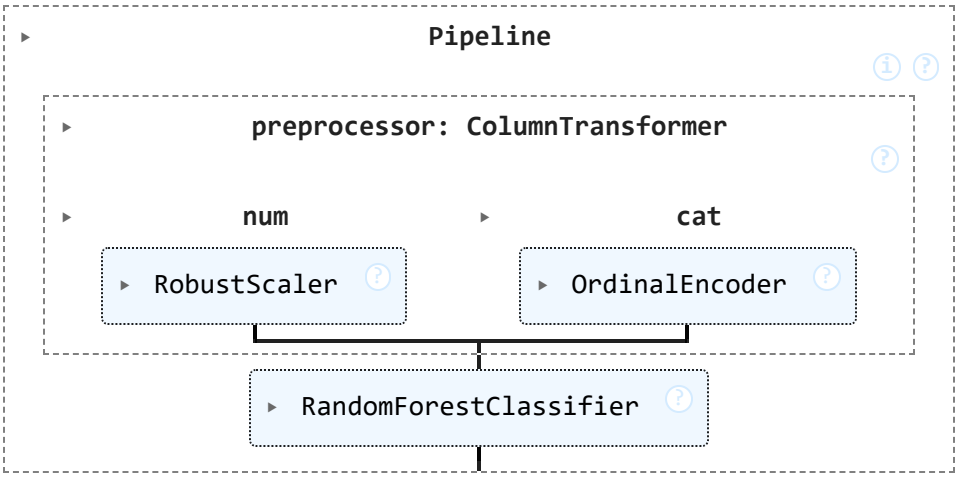


9. **Train the Random Forest**

- Fit the pipeline using `X_train` and `y_train`.

In [23]:
```
full_pipeline.fit(X_train, y_train)
```

Out[23]:



10. **Make predictions**

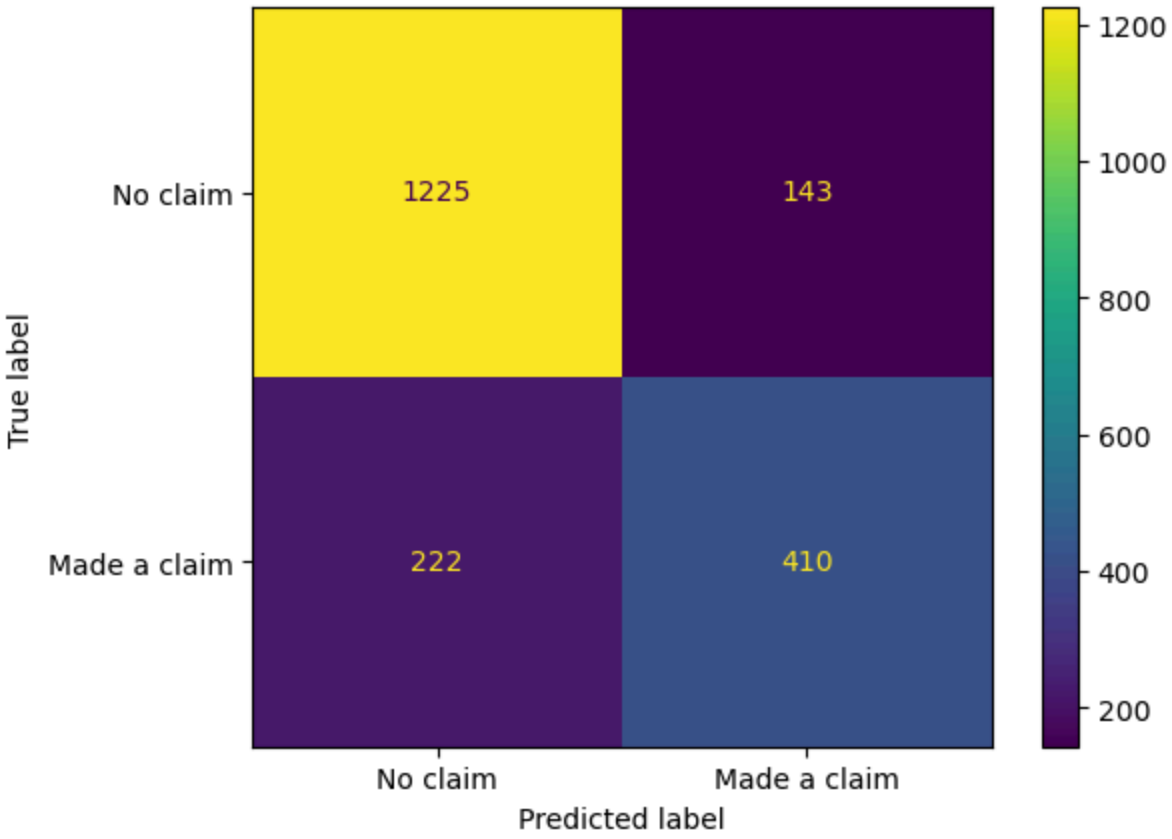
- Predict `X_test`.
- Store the result as `y_pred`.

In [24]:
```
y_pred = full_pipeline.predict(X_test)
```

11. **Evaluate the model**

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1-score
- Classification Report

In [25]:
```
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['No claim', 'Mad
disp.plot()
plt.show()
```



In [26]:
```
df['outcome'].value_counts()
```

Out[26]:
```
outcome
0.0    6867
1.0    3133
Name: count, dtype: int64
```

In [27]:
```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

         0.0       0.85      0.90      0.87      1368
         1.0       0.74      0.65      0.69       632

    accuracy                           0.82      2000
   macro avg       0.79      0.77      0.78      2000
weighted avg       0.81      0.82      0.81      2000
```

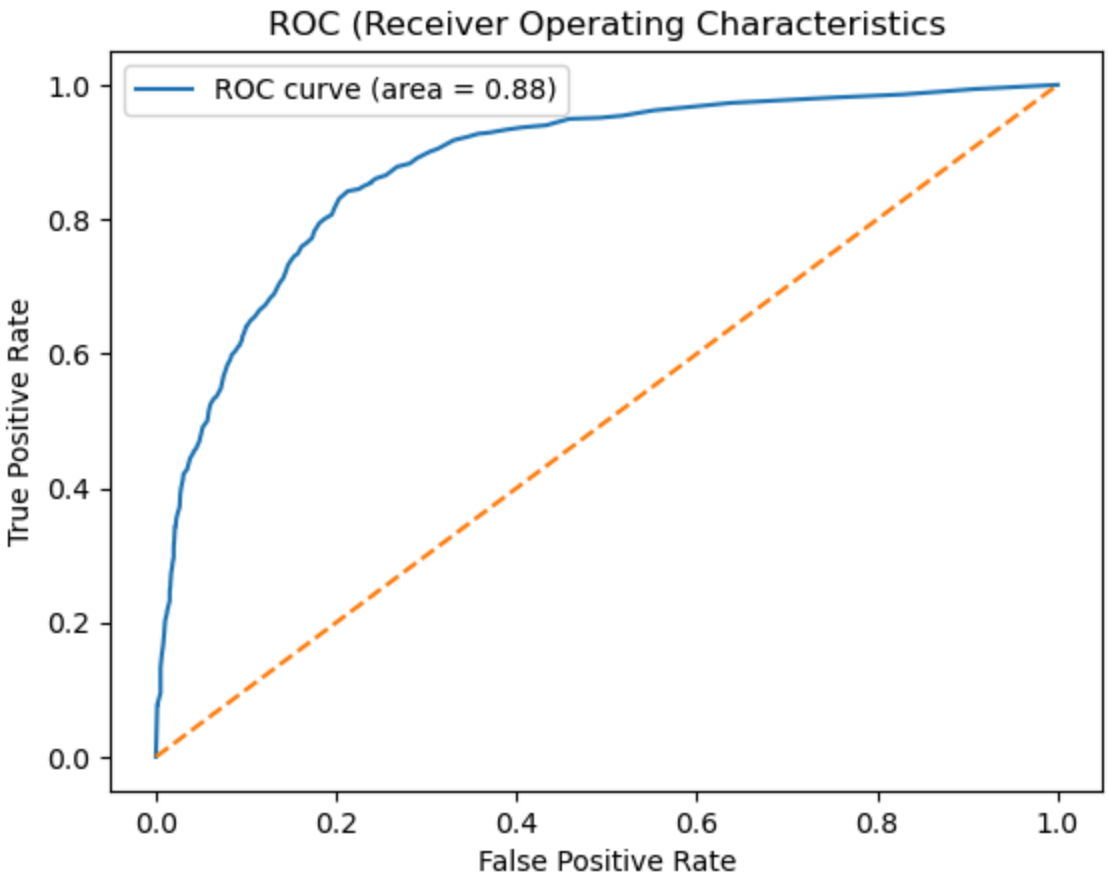
- ROC and AUC

In [28]:
```
from sklearn.metrics import roc_curve, auc
```

In [29]:
```
y_proba = full_pipeline.predict_proba(X_test)[:,1]
```

In [30]:
```
fpr, tpr, thresholds = roc_curve(y_test, y_proba)
roc_auc = auc(fpr, tpr)
```

In [31]:
```
plt.figure()
plt.plot(fpr, tpr, label='ROC curve (area = %0.2f)' % roc_auc)
plt.plot([0,1], [0,1], linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC (Receiver Operating Characteristics')
plt.legend()
plt.savefig('roc.png')
plt.show()
```



Hyperparameter tuning

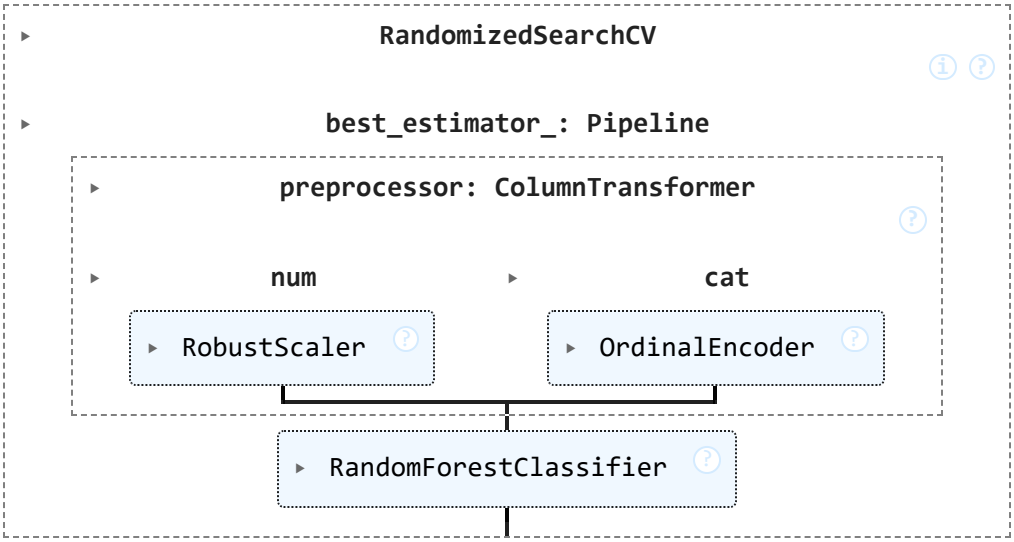
In [32]:
```
param_dist = {
    "classifier__max_depth": [3, 5, 7, 10, None],
    "classifier__min_samples_split": [2, 5, 10],
    'classifier__min_samples_leaf': randint(1,10),
    "classifier__criterion": ["gini", "entropy", "log_loss"],
    'classifier__n_estimators': randint(50, 200)
}

random_search = RandomizedSearchCV(
    estimator=full_pipeline,
    param_distributions=param_dist,
    cv=5, scoring='accuracy', random_state=42, n_jobs=-1, verbose=1
)

random_search.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 10 candidates, totalling 50 fits
```

Out[32]:



In [33]:
```
random_search.best_params_
```

Out[33]:
```
{'classifier__criterion': 'log_loss',
 'classifier__max_depth': 10,
 'classifier__min_samples_leaf': 8,
 'classifier__min_samples_split': 2,
 'classifier__n_estimators': 70}
```

In [34]:
```
random_search.best_score_
```

Out[34]:
```
np.float64(0.837)
```

In [35]:
```
rf_tuned = random_search.best_estimator_
```

In [36]:
```
y_pred_tuned = rf_tuned.predict(X_test)
y_pred_tuned_proba = rf_tuned.predict_proba(X_test)[:, 1]
```

In [37]:
```
roc_auc_score(y_test, y_proba)
```

Out[37]:
```
0.8804870826856169
```

In [38]:
```
roc_auc_score(y_test, y_pred_tuned_proba)
```

Out[38]:
```
0.8962184932267376
```

In [39]:
```
confusion_matrix(y_test, y_pred) # normal
```

Out[39]:
```
array([[1225,  143],
       [ 222,  410]])
```

In [40]:
```
confusion_matrix(y_test, y_pred_tuned) # with tuner
```

Out[40]:
```
array([[1234,  134],
       [ 198,  434]])
```

12. **Write a conclusion**

- Explain the preprocessing performed.
- State the model's performance.
- Interpret the evaluation metrics.
- Conclude how well the Random Forest predicts insurance claims.

1. Explain the preprocessing performed.
    - Filled missing values (in credit_score and annual_mileage, both around 10%) with the mean value using SimpleImputer.
    - Scaled number columns using RobustScaler.
    - Turned category columns into numbers using OrdinalEncoder.
    - Split data into 80% for training, 20% for testing with train_test_split.
2. State the model's performance.
    - Accuracy: 81% - it got the right answer 81 out of 100 times.
    - ROC AUC: 0.88 - a strong score (1.0 = perfect, 0.5 = random guessing).
3. Interpret the evaluation metrics.
    - The model is good at spotting "no claim" customers (89% recall - catches most of them).
    - The model is weaker at spotting "made a claim" customers (65% recall- misses about 1 in 3 real claims).
    - This means the model plays it safe: it correctly flags most non-risky clients, but lets some risky ones slip through.
4. Conclude how well the Random Forest predicts insurance claims.
    - The Random Forest model does a decent job overall (81% accuracy, strong AUC), but it's better at confirming who won't claim than catching who will claim.

In [41]:
```
from pathlib import Path

import joblib

MODEL_PATH = "full_pipeline.joblib"

bundle = {"model": full_pipeline, "labels": ['No claim', 'Made a claim']}

# Saving the model
joblib.dump(bundle, MODEL_PATH)
print(f"{Path(MODEL_PATH).stat().st_size/1e6:.2f} MB")
```

```
25.45 MB
```

In [42]:
```
bundle = joblib.load("full_pipeline.joblib")
```

In [43]:
```
model = bundle["model"]
LABELS = bundle["labels"]
```

```
FEATURES = list(model.feature_names_in_)

def predict_car(data):
    df = pd.read_csv(data) if isinstance(data, str) else data.copy()
    missing = [c for c in FEATURES if c not in df.columns]
    if missing:
        raise ValueError(f"Input is missing required columns(s): {missing}")
    X_new = df[FEATURES]
    proba = model.predict_proba(X_new)
    output = df.copy()
    output["outcome"] = proba.max(axis=1)
    return output
```

In [47]:
```
df.loc[1,:]
```

Out[47]:
```
id                          750365
age                              0
gender                           1
driving_experience             0-9y
education                      none
income                      poverty
credit_score               0.357757
vehicle_ownership               0.0
vehicle_year           before 2015
married                         0.0
children                        0.0
postal_code                   10238
annual_mileage              16000.0
vehicle_type                  sedan
speeding_violations               0
duis                              0
past_accidents                    0
outcome                         1.0
Name: 1, dtype: object
```

In [ ]:
```
import gradio as gr

DEFAULTS = {
    "age": 0,
    'gender': 1,
    "driving_experience": '0-9y',
    "education": 'none',
    "income": 'poverty',
    "credit_score": 0.357757,
    "vehicle_ownership": 0,
    "vehicle_year": 'before 2015',
    "married": 0,
    "children": 0,
    "annual_mileage": 16000.0,
    "vehicle_type": 'sedan',
    "speeding_violations": 0,
    "duis": 0,
    "past_accidents": 0,
}

CATEGORICAL_OPTIONS = {
```

```
    'age': [('16-25', 0), ('26-39', 1), ('40-64', 2), ('65+', 3)],
    'gender': [('Female', 0), ('Male', 1)],
    'driving_experience': ['0-9y', '10-19y', '20-29y', '30y+'],
    'education': ['high school', 'none', 'university'],
    'income': ['upper class', 'poverty', 'working class', 'middle class'],
    'vehicle_year': ['after 2015', 'before 2015'],
    'married': [('Not married', 0), ('Married', 1)],
    'vehicle_type': ['sedan', 'sports car'],
}

def build_input(f):
    if f == 'credit_score':
        return gr.Slider(label=f, value=DEFAULTS[f], minimum=0.0, maximum=1.0)
    if f in CATEGORICAL_OPTIONS:
        return gr.Dropdown(choices=CATEGORICAL_OPTIONS[f], value=DEFAULTS[f], label
    return gr.Number(label=f, value=DEFAULTS[f])

def classify(*values):
    row = pd.DataFrame([dict(zip(FEATURES, values))])
    proba = model.predict_proba(row)[0]
    return {label: float(p) for label, p in zip(LABELS, proba)}

demo = gr.Interface(
    fn=classify,
    inputs=[build_input(f) for f in FEATURES],
    outputs=gr.Label(num_top_classes=2, label='Predicted Outcome'),
    title='Insurance Outcome Prediction',
)

demo.launch(debug=True)
```

```
* Running on local URL:  http://127.0.0.1:7862
* To create a public link, set `share=True` in `launch()`.
```

In [ ]: